# CloudGoat Attack Scenario Writeups

*Phase 6, Weeks 23-24 — Cloud Security Study Plan. Three scenarios deployed via Terraform in a dedicated, isolated AWS account/IAM boundary, executed, and fully torn down after each.*

## Scenario 1 — iam_privesc_by_attachment

Starting identity: kerrigan, a low-privilege IAM user.

### Objective

Escalate from a limited IAM user to full administrative access, then delete a target resource (a simulated "super critical security server") to prove impact.

### What We Did

1. Enumerated kerrigan's permissions blind — no direct policy-read access, so tested individual actions to see what succeeded vs. was denied.
2. Found kerrigan could list IAM roles and instance profiles, and — critically — modify which role an existing instance profile points to (iam:AddRoleToInstanceProfile, iam:RemoveRoleFromInstanceProfile, iam:PassRole).
3. Identified two roles: a harmless "meek" role (deny-all) and a highly privileged "mighty" role (Allow: *, Resource: *), each with its own instance profile.
4. Removed the meek role from its instance profile, then attached the mighty role to that same instance profile — an instance profile can only hold one role at a time, so the swap required both steps.
5. Launched a new EC2 instance using the now-repurposed instance profile. The instance automatically inherited the mighty role's permissions via the EC2 metadata service (IMDS).
6. SSH'd into the new instance, queried IMDS (curl http://169.254.169.254/latest/meta-data/iam/security-credentials/<role>) to retrieve temporary admin credentials.
7. Used those credentials to terminate the target "super critical security server" instance, completing the scenario objective.
8. Cleanup: manually terminated the two EC2 instances created during the attack (not tracked by CloudGoat's Terraform state) and deleted the manually-created key pair, then ran cloudgoat destroy for the 16 CloudGoat-tracked resources.

### Root Cause

kerrigan's policy combined iam:PassRole with instance-profile modification rights, scoped to Resource: "*" — meaning she could point any instance profile at any role in the account, including ones far more privileged than her own. No single permission was dangerous alone; the combination created the escalation path.

### Prevention / Solution

- Scope iam:PassRole tightly using conditions (e.g. iam:PassedToService, or an explicit list of allowed role ARNs) instead of granting it against Resource: "*".

- Avoid granting instance-profile-modification permissions (AddRoleToInstanceProfile / RemoveRoleFromInstanceProfile) to routine or low-trust identities — this is an infrastructure-level action, not a typical developer need.
- Never create a role with Action: "*", Effect: Allow, Resource: "*" if avoidable. An overly powerful role sitting unused in an account is a standing risk waiting for an indirect path to reach it.
- Apply least privilege to what a role can indirectly reach, not just what a user can do directly — role-passing and resource-attachment permissions are a common and often-overlooked escalation surface.

## Scenario 2 — cloud_breach_s3

Starting identity: anonymous outsider — no credentials, no prior access.

### Objective

Exploit a misconfigured reverse proxy to reach the EC2 metadata service, steal the attached IAM role's credentials, and exfiltrate sensitive data from an S3 bucket.

### What We Did

9. Sent a plain HTTP request to the deployed nginx reverse-proxy server. The response explicitly revealed the misconfiguration: "This server is configured to proxy requests to the EC2 metadata service. Please modify your request's 'host' header and try again."
10. Sent a follow-up request with the Host header set to 169.254.169.254 (the EC2 metadata service IP), which the proxy blindly forwarded — a classic Server-Side Request Forgery (SSRF).
11. Queried IMDS through the proxy to list available IAM roles, then retrieved temporary credentials for the attached role (cg-banking-WAF-Role, granted AmazonS3FullAccess) — all without ever touching the instance directly via SSH.
12. Exported the stolen credentials locally and ran aws s3 ls, which revealed not just the scenario's intended bucket but also the AWS account's own unrelated CloudTrail log buckets — a side effect of the role's overly broad managed policy.
13. Listed and downloaded files from the target bucket (cardholder_data_primary.csv, cardholder_data_secondary.csv, cardholders_corporate.csv, goat.png), completing the data exfiltration objective.
14. Saved one exfiltrated file locally as evidence, then cleaned up: ran cloudgoat destroy, which removed all 18 tracked resources (no untracked resources were created this time, since no new EC2 instance or key pair was needed).

### Root Cause

The reverse proxy trusted a user-controlled input (the HTTP Host header) to decide which internal address to forward requests to, with no allowlist or validation. Combined with IMDSv1 (no token requirement) and an overly broad IAM role attached to the instance, a single unauthenticated request chain led directly to sensitive data exposure.

### Prevention / Solution

- Enforce IMDSv2 (require a session token via a PUT request before any metadata GET succeeds) on every EC2 instance. Most SSRF vulnerabilities only allow an attacker to influence a simple GET request, so IMDSv2 blocks this exact attack path even if the proxy misconfiguration still exists — this is the single most effective fix, and is already applied on our own Docker host (metadata_options { http_tokens = "required" }).
- Set http_put_response_hop_limit = 1 to further restrict how many network hops a metadata request can traverse.
- Never let a proxy or gateway forward requests based on unvalidated user input (e.g. the Host header) — allowlist only legitimate backend destinations.
- Scope IAM roles attached to internet-facing instances narrowly (specific bucket ARNs, not AmazonS3FullAccess across the whole account) to limit blast radius even if SSRF succeeds.
- This exact pattern — SSRF into cloud metadata leading to a major data breach — is not hypothetical: it is fundamentally how the 2019 Capital One breach (100M+ records) occurred.

# Scenario 3 — lambda_privesc

Starting identity: chris, a limited IAM user.

## Objective

Escalate from a limited IAM user to full administrative access by abusing Lambda function creation and role-passing permissions.

## What We Did

15. Enumerated chris's permissions: sts:AssumeRole, iam:List*, iam:Get* — broad enough to discover roles, but not to act on most of them directly.
16. Listed IAM roles and found two of interest: cg-debug-role (trusts only the Lambda service, attached to AdministratorAccess) and cg-lambdaManager-role (grants lambda:* and iam:PassRole).
17. Attempted to assume cg-debug-role directly — denied, since its trust policy only allows the Lambda service as principal, not IAM users.
18. Assumed cg-lambdaManager-role instead, which chris's policy did permit. This granted lambda:* and iam:PassRole.
19. Wrote a minimal Lambda function (Python) that calls sts.get_caller_identity() and returns its own execution role's temporary credentials.
20. Created the Lambda function, explicitly specifying cg-debug-role as its execution role — permitted because cg-debug-role's trust policy allows the Lambda service, and chris (via the assumed lambdaManager role) held iam:PassRole to authorize the pairing.
21. Invoked the function. Its response contained fresh credentials scoped to assumed-role/cg-debug-role — full admin.
22. Verified the escalation was genuine (not just self-reported) by running aws iam list-users with the stolen credentials — an action chris's original policy never granted — and it succeeded, returning every IAM user in the account.
23. Saved the Lambda invocation output and the list-users result as evidence, then cleaned up: deleted the manually-created Lambda function (untracked by Terraform), then ran cloudgoat destroy for the 9 CloudGoat-tracked resources.

### Root Cause

chris could not assume the admin role directly, but could assume an intermediate role holding both lambda:* and iam:PassRole with no restriction on which roles could be passed. Since AWS Lambda's own trust relationship allowed the admin role to be used as any function's execution role, creating a Lambda function became an indirect, fully permitted path to admin — the classic "PassRole + resource-creation permission" privilege escalation pattern, same underlying category as Scenario 1 but via a different AWS service.

### Prevention / Solution

- Scope iam:PassRole with a condition restricting which specific role ARNs can be passed (e.g. iam:PassedToService combined with a role ARN allowlist), rather than granting it broadly alongside a resource-creation permission like lambda:*.
- Apply the principle that any identity able to both create a compute resource (Lambda, EC2, etc.) AND pass an arbitrary role to it should be treated as equivalent in privilege to whatever role it could pass — audit these combinations specifically, not just individual permissions in isolation.
- Restrict which roles a given service (Lambda, EC2, ECS) can be assumed by at the trust-policy level as narrowly as practical, and pair that with tightly scoped PassRole grants on the human/service identity side.
- Monitor and alert on Lambda function creation events referencing highly-privileged execution roles (e.g. via CloudTrail + a detective control) — this is a fast, low-noise way to catch this exact technique in a real environment.

## Cross-Cutting Observations

All three scenarios ultimately relied on the same underlying pattern despite touching different AWS services (EC2/IAM, S3/networking, Lambda/IAM): a low-privilege identity held a narrow, seemingly reasonable permission that — combined with iam:PassRole or equivalent trust-based indirection — allowed it to reach a resource or role far more privileged than its own direct grants suggested.

- Least privilege must account for indirect reach (role-passing, resource attachment, trust relationships), not just an identity's own direct action list.
- IMDSv2 enforcement is a cheap, high-value control that would have fully blocked Scenario 2 even with the proxy misconfiguration still present.
- Overly broad managed policies (AmazonS3FullAccess, AdministratorAccess) attached to service roles dramatically increase blast radius the moment any single link in a chain is compromised.
- In every scenario, cleanup required tracking resources created manually during the attack (EC2 instances, key pairs, a Lambda function) separately from what CloudGoat's own Terraform state tracked — a reminder that attacker-created resources in a real incident are exactly the kind of thing that gets missed during containment.

*Milestone: 3 CloudGoat scenarios completed and documented, meeting and exceeding the Weeks 23-24 target of 2.*